

“초등부 2번 / 고등부 1번. 대피소” 문제 풀이

작성자: 오주원

부분문제 1

대피소를 설치하지 않을 집을 선택한 경우, 해당 집과 가장 가까운 집 사이의 거리가 대피소로 이동하기 위한 최대 거리가 된다. 따라서, 문제의 정답을 찾기 위해서는 각 집마다 가장 가까운 집과의 거리를 계산하고, 이 중에서 최솟값을 찾아야 한다. 즉, $\frac{N(N-1)}{2}$ 개의 거리 쌍 중에서 가장 작은 거리가 문제의 정답이 된다.

부분문제 2

어떤 집에 대피소를 설치하더라도 가장 먼 집은 1번 집 또는 N 번 집이므로 대피소와 두 집 사이의 거리만 고려하면 된다. 대피소를 K 번 집에 설치하면 최대 거리는 $N - K$ 와 $K - 1$ 중 더 큰 값이 된다. 찾아야 하는 것은 이 최대 거리를 최소화하는 K 이다. 중앙에 가까운 집을 선택할수록 최대 거리가 줄어드는 것을 알 수 있고, 따라서 가장 중앙에 가까운 집을 선택했을 때의 거리가 최소가 되어 $\lfloor \frac{N}{2} \rfloor$ 가 문제의 정답이 된다.

부분문제 3

대피소를 설치할 집 하나를 선택한 다음, 설치하지 않은 다른 집들과의 거리를 비교하여 가장 큰 값을 찾는다. 총 N 개의 집을 선택할 수 있고, 최대 거리를 구하는 데에 $O(N)$ 의 시간이 걸리므로 $O(N^2)$ 에 정답을 구할 수 있다.

부분문제 4

대피소를 설치할 집 하나를 선택하고, 설치하지 않은 집 중에서 하나를 골라 대피소를 설치한다. 그다음, 각 집과 두 대피소와의 거리를 비교해서 더 작은 값으로 정답을 갱신해준다. N 개의 집 중에 2개의 집을 선택하는 방법은 $\frac{N(N-1)}{2}$ 개가 있고, 거리를 구하는 데 $O(N)$ 의 시간이 소요된다. 따라서, $O(N^3)$ 의 시간복잡도로 정답을 구할 수 있다.

부분문제 5

대피소를 1번 집과 N 번집에 설치하면 최대 거리는 $X_N - X_1$ 보다 작아진다는 것을 알 수 있다. 따라서 0부터 $X_N - X_1$ 까지 1씩 증가시키며 하나씩 정답으로 가능한지 확인한 이후, 가능하면 그 값을 바로 출력해주고 프로그램을 종료하는 것으로 항상 정답을 찾을 수 있다.

정답으로 A 가 가능한지 확인하기 위해서 탐욕적인 방법을 사용한다. 가장 작은 번호의 집에 설치할 대피소는 1번 집과의 거리가 A 이하여야 하고, 그중 가장 번호가 큰 집을 선택해야 그 대피소와 거리가 A 이하인 집의 개수가 가장 많아진다. 다음으로 중간에 설치할 대피소는 첫 번째로 설치한 대피소와 거리가 A 보다 크면서 번호가 가장 작은 집을 z 라고 할 때, z 와의 거리가 A 이하여야 한다. 앞과 마찬가지로 이유로, z 와 거리가 A 이하이면서 가장 번호가 큰 집에 대피소를 설치해야 하고, 같은 방법으로 세 번째 대피소를 설치할 집도 찾아주면 된다. 이렇게 대피소를 설치하면 1번 집부터 세 번째 대피소를 설치한 집까지는 모두 대피소와의 거리가 A 이하가 되며, 세 번째 대피소를 설치한 집보다 번호가 크면서 가장 멀리 떨어진 집은 N 번 집이므로 두 집 사이의 거리를 구하여 모든 거리가 A 이하인지 확인할 수 있다. 이때, 모든 거리가 A 이하라면 A 는 정답이 될 수 있고, 없다면 A 는 정답이 될 수 없다.

1부터 X_N 까지 각 값을 가능한지 확인하는데 $O(K + N)$ 의 시간이 걸리므로 $O((K + N)X_N)$ 에 문제를 해결할 수 있다. 추가로, 이분 탐색을 이용해 $O(K \log X_N \log N)$ 에 문제를 해결할 수도 있다.

부분문제 6

대피소를 설치할 세 집을 선택하고, 선택한 세 대피소와 설치하지 않은 집 간의 거리를 비교하여 가장 작은 값의 최댓값이 정답의 후보가 된다. N 개의 집 중에 3개의 집을 선택하는 방법은 $\frac{N(N-1)(N-2)}{6}$ 개가 있으며, 거리를 구하는 데 $O(N)$ 의 시간이 걸리므로 $O(N^4)$ 에 정답을 구할 수 있다.

“중등부 3번 / 고등부 2번. 주유소” 문제 풀이

작성자: 김준원

부분문제 1

부분문제의 조건과 함께 문제를 해석하면, 선형으로 연결되어 있는 N 개의 마을 가운데 연속해서 $k + 1$ 개의 마을에 모두 주유소가 없는 경우가 없도록 마을을 고르는 문제가 된다.

$(k + 1)$ 의 배수)번 마을에 주유소를 하나씩 설치하는 방법을 생각할 수 있다. 그리고 이 방법이 가장 적은 개수의 마을을 골라 주유소를 설치하는 방법이다. 따라서 $\lfloor \frac{N}{k+1} \rfloor$ 를 출력하면 된다.

부분문제 4

앞으로의 풀이에는 그래프 이론의 용어를 사용한다. 즉, KOI 국가는 트리 형태이며, 마을은 정점, 두 마을을 잇고 있는 도로는 간선으로 표현한다.

주유소를 고를 수 있는 가능한 2^N 가지의 배치를 각각 고려한다. 그래프에서 길이가 K 인 모든 경로를 확인하면서, 각 배치가 가능한 배치인지 판별할 수 있다. 트리에서 두 정점을 잇는 길이 k 의 경로는 N^2 개 이하 존재하므로, 시간 복잡도는 $O(2^N N^2 k)$ 이다.

부분문제 2

부분문제 2에서 간선으로 연결되어 있는 두 정점 가운데 하나 이상에는 주유소가 설치되어 있어야 한다. ‘그리디’하게, 즉 어쩔 수 없을 때에만 주유소를 설치하도록 하는 알고리즘을 통해 목표를 이룰 수 있다.

임의의 정점을 루트로 잡은 뒤 DFS를 수행하자. 그러면 간선을 타고 내려가 리프 정점 (더 이상 타고 내려갈 간선이 없는 정점)에 도달한 뒤 다시 올라오는 과정에 전략을 수행할 수 있다.

- 리프 정점에는 주유소를 설치하지 않는다.
- 내부 정점 v 에 대해, v 의 자녀 정점들에 주유소가 하나도 설치되어 있지 않다면 주유소를 설치한다.

$O(N)$ 의 시간 복잡도로 부분문제 2를 풀 수 있다. 이는 아래와 같이 전체 문제에 대한 풀이로 확장할 수 있다.

부분문제 5, 6

앞선 부분문제 2에서의 최적 전략은, ‘자녀 정점에 주유소가 설치되어 있지 않을 때’ 주유소를 설치하는 것이다. 여기서 일반적인 경우의 최적 전략을 추론할 수 있다. 역시 주유소를 ‘그리디’한 방법으로 배치하는 전략이다.

임의의 정점을 루트로 잡은 뒤 DFS를 수행한다. 이제 다음과 같은 조건을 만족하는 정점 v 에 주유소를 설치하면 된다.

- v 의 각 자녀 정점 w 에 대해, w 를 루트로 하는 서브트리에는 주유소가 설치되어 있지 않은 길이가 k 인 경로가 존재하지 않는다.
- v 를 루트로 하는 서브트리에는 주유소가 설치되어 있지 않은 길이가 k 인 경로가 존재한다.

위 전략을 시행하려면, 어떤 정점 v 에 대해 ‘ v 를 루트로 하는 서브트리에서 주유소가 설치되어 있지 않은 가장 긴 경로의 길이’를 계산할 수 있어야 한다. 이를 직접 (비효율적으로) 계산하면 시간 복잡도에 따라 부분문제 5, 6을 풀 수 있다. 이를 빠르게 계산하면 부분문제 7을 풀 수 있다.

부분문제 3, 7

어떤 정점 v 를 루트로 하는 서브트리에서 가장 긴 경로는 어떤 경로일까? 만약 v 의 자녀 정점이 없다면 가장 긴 경로의 길이는 0일 것이다. v 의 자녀 정점이 하나 있다면, v 에서 시작해서 이 자녀 정점을 지나 가장 깊은 리프 정점에서 끝나는 경로가 가장 긴 경로가 될 것이다. v 의 자녀 정점이 두 개 이상이라면, ‘가장 깊은 리프 정점의 깊이’가 가장 깊은 자녀 정점 두 개를 경유하는 경로, 즉 해당 두 리프 정점을 잇는 경로가 될 것이다.

따라서 가장 긴 경로를 찾기 위해서는, 먼저 각 정점에 대해 ‘이 정점 아래에 있는 가장 깊은 리프 정점의 깊이’를 계산하여야 한다. DFS를 수행하면서 해당 정보를 먼저 계산해 두면, 위 문단의 방법에 따라 가장 긴 경로의 길이도 알 수 있다. 정점 v 에 대해 각 자녀 정점에서 가장 긴 경로의 길이가 k 미만인데 v 에서 가장 긴 경로의 길이가 k 이상이 된다면, 정점 v 에 주유소를 설치한다. v 에 주유소를 설치하고 나면, 다른 정점들에 대해 경로를 계산할 때 v 를 제외해야 한다. 주유소가 설치되어 있지 않은 경로만 생각해야 하기 때문이다. 해당 처리도 v 에 대한 DFS 함수가 반환하고 부모 정점으로 올라가는 시점에 수행할 수 있다. 총 시간 복잡도는 $O(N)$ 이 된다.

“고등부 3번. 블록 쌓기” 문제 풀이

작성자: 이종영

부분문제 1

$A_1 + \dots + A_N < LN$ 이거나 $A_1 + \dots + A_N > RN$ 인 경우 목표를 달성할 수 없다. 그렇지 않은 경우 항상 목표를 달성할 수 있다.

$L = R$ 인 경우 자명히 가능한 최종 상태의 수가 하나이다. $L + 1 = R$ 인 경우, $L + 1$ 의 개수에 따라 합이 달라지므로 이번에도 가능한 최종 상태는 하나이다. 이 경우의 최소 비용은 블록들의 순서를 유지하는 그리디 알고리즘으로 구할 수 있다. 첫 칸부터 순서대로 해당 칸에 원하는 만큼 블록을 쌓을 때까지 아직 최종 위치가 결정되지 않은 가장 왼쪽의 블록을 해당 칸까지 옮기면 된다. 이 때, 한 번에 가능한 최댓값만큼 블록을 동시에 옮겨 주면 $O(N)$ 에 비용을 구할 수 있다.

부분문제 2

$O((R - L)^N)$ 가지의 가능한 최종 상태들을 고려한다. 각 상태에 대해서는 부분문제 1과 동일하게 최소 비용을 계산하자. 이 중 최솟값이 답이 된다.

부분문제 3

가능한 상태들의 수가 적다. $N = 10, A_1 + \dots + A_N = 10$ 인 경우의 수는 총 92378개가 존재한다. 이는 중복조합 등을 이용해 수학적으로 증명하거나, 백트래킹 등 프로그래밍을 이용해서도 확인할 수 있다.

가능한 상태들의 수가 많지 않으므로, 한 상태에서 블록을 옮기는 연산을 통해 다른 상태로 이동하는 것을 BFS로 구현해주면 된다.

부분문제 4

최종 배열을 B 라고 하자. B_i 가 단조증가한다는 점에서 B 는 $A_1 + \dots + A_N$ 의 분할에 대응한다고 생각할 수 있다. 50의 분할 수는 204226으로, 결국 가능한 최종 상태의 수가 적음을 알 수 있다.

모든 가능한 B 배열을 순회하는 것은 백트래킹을 이용해 구현할 수 있다. 각 최종 상태에 대한 비용은 부분문제 1과 동일하게 계산할 수 있다.

부분문제 5

$D_{i,v,s}$ 를 B 의 첫 i 개 값을 결정한 상태에서, $B_i \leq v$ 이며 $B_1 + \dots + B_i = s$ 일 때, 최소 비용으로 정의하자.

$D_{i,v,s}$ 는 $B_i = v$ 인 경우 $D_{i-1,v,s-v}$ 에서, $B_i < v$ 인 경우 $D_{i-1,v-1,s}$ 에서 전이가 가능하다. 그리디 알고리즘에서 첫 칸부터 순서대로 채워나가는 과정을 DP 전이 과정에서 전이 비용의 계산에 사용할 수 있다.

총 상태의 수가 $O(N^2 R^2)$ 개이며, 각 전이 비용의 계산에 $O(N)$ 의 시간이 필요하므로 $O(N^3 R^2)$ 에 문제를 해결할 수 있다.

부분문제 6

$L \leq v \leq R, iL \leq s \leq iR$ 를 만족하므로, 가능한 (i, v, s) 의 개수는 $O(N^2(R - L)^2)$ 개이다. 해당 상태들만 고려하도록 부분문제 5의 풀이를 개선하면 $O(N^3(R - L)^2)$ 에 문제를 해결할 수 있다.

부분문제 7

A 의 부분합 배열을 S , B 의 부분합 배열을 T 라고 하자. 각 i 에 대해 i 와 $i+1$ 번 칸 사이의 이동의 횟수를 생각하자. 첫 상태에서는 i 번 칸까지 S_i 개의 블록이 있는 반면, 최종 상태에서는 T_i 개의 블록이 있다. 따라서 $S_i < T_i$ 라면 i 번째 칸으로 $T_i - S_i$ 개의 블록이 이동했어야 하며, $S_i > T_i$ 라면 $i+1$ 번째 칸으로 $S_i - T_i$ 개의 블록이 이동했어야 한다. 반면, 양 방향의 이동이 모두 있었다면, 두 이동을 하나씩 실행하지 않아도 결과가 같다. 따라서 i 와 $i+1$ 번 칸 사이의 이동의 최소값은 $|S_i - T_i|$ 이며, 최적해에서는 실제로 해당 수 만큼의 이동이 이루어진다. 따라서 최소 비용은 $\sum_{i=1}^{N-1} |S_i - T_i|$ 이다. $S_N = T_N$ 이므로 $\sum_{i=1}^N |S_i - T_i|$ 으로도 생각할 수 있다.

이 사실을 이용하면 부분문제 6과 같은 DP 정의에서, 전이 비용을 계산을 $O(1)$ 에 할 수 있다. 따라서 문제를 $O(N^2(R-L)^2)$ 에 해결할 수 있다.